Setup - Redis & RedisInsight

Setting up Redis on a local machine used to mean compiling C binaries or installing native services directly onto your operating system. In modern software engineering, you should never pollute your host machine with database engines or background services.

The industry standard is to spin up isolated, throwaway containers using **Docker** and **Docker Compose**, paired with a visual GUI tool called **RedisInsight**.

1. Why Docker Over Native Installation?

- **Zero Environment Drift:** A Redis instance running inside Docker on your development laptop behaves identically to the Redis instance running in production on AWS, GCP, or Azure.
- **Instant Version Switching:** Need to test an upgrade from Redis 7 to Redis 8? Change a single line in a configuration file instead of uninstalling packages.
- **Complete Teardown & Reset:** Corrupted your cache or filled memory during testing? Run one command to wipe the container and start fresh in seconds.
- **No Host Conflicts:** Redis uses default port `6379`. Running it in a container avoids port collisions with other tools or services on your machine.

2. The Quick Run: `docker run`

For quick, temporary tasks, a single CLI command spins up an active Redis container:

```
docker run -d \
  --name local-redis \
  -p 6379:6379 \
  redis:7.4-alpine
```

Breakdown of Flags

- `-d` (Detached mode): Runs the container in the background, freeing up your terminal.
- `--name local-redis`: Gives the container an easily identifiable name instead of a random Docker-generated hash.
- `-p 6379:6379`: Binds port `6379` of the container to port `6379` on your `localhost`.
- `redis:7.4-alpine`: Pulls the lightweight Alpine Linux build (~30–40 MB), minimizing download time and disk usage.

> While `docker run` is fine for quick experiments, production-grade applications require multi-service setups (Redis engine + Visual Dashboard + Persistence). For that, we use **Docker Compose**.

3. Production-Ready Local Stack: `docker-compose.yml`

Create a file named `docker-compose.yml` in your project root. This orchestrates both the Redis engine and the **RedisInsight** management GUI in a private network.

```yaml
services:
  redis:
    image: redis:7.4-alpine
    container_name: redis-cache
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    command: >
      redis-server
      --save 60 1
      --loglevel notice
      --requirepass devpassword123

  redis-insight:
    image: redis/redisinsight:latest
    container_name: redis-insight
    restart: unless-stopped
    ports:
      - "5540:5540"
    depends_on:
      - redis
    volumes:
      - redis-insight-data:/data

volumes:
  redis-data:
    driver: local
  redis-insight-data:
    driver: local
```

Architectural Decisions in this Configuration

1. **Volume Mounts (`redis-data:/data`):**
   - Containers are stateless by default; destroying a container deletes all data.
   - Mounting a Docker named volume guarantees that your local cache keys and RDB snapshots survive container restarts and system reboots.
2. **Snapshot Rule (`--save 60 1`):**
   - Tells Redis: *"If at least 1 key changes, write an RDB snapshot to disk every 60 seconds."* Useful for local development persistence.
3. **Authentication (`--requirepass devpassword123`):**

- Simulates production behavior. Hardcoding empty passwords in local dev often leads to security vulnerabilities when deploying to staging or production.

4. **RedisInsight Integration on Port `5540`:**
   - Runs the official web-based Redis visualizer without requiring desktop software installation.

4. Starting and Managing the Stack

Launch the Services

From the directory containing your `docker-compose.yml`:

```
docker compose up -d
```

Check Container Status

Verify that both containers are running and healthy:

```
docker compose ps
```

View Real-Time Logs

Watch Redis operations and startup output in real time:

```
docker compose logs -f redis
```

Stop the Services

Pause the containers without deleting cached data:

```
docker compose down
```

Complete Reset (Wipe All Cache Data)

Stop containers and destroy all volume data:

```
docker compose down -v
```

5. Visualizing the Engine with RedisInsight

Once the containers are running, navigate to:

```
http://localhost:5540
```

Connecting RedisInsight to Your Container

1. Click **"Add Redis Database"**.
2. Set the connection properties:
   - **Host:** `redis` (if using Docker network resolution) or `host.docker.internal` / `localhost` (when accessing mapped ports).
   - **Port:** `6379`
   - **Username:** `default`
   - **Password:** `devpassword123`
   - **Database Alias:** `Local-Dev-Redis`
3. Click **"Add Database"**.

6. What RedisInsight Gives You (Beyond Terminal CLI)

- **Key Browser with Live Filters:** Search across keys using patterns (e.g., `users:*`, `chat:*`) without running slow, blocking `KEYS *` scans.
- **Visual TTL Countdown:** Watch key expiration counters tick down to zero in real time.
- **Memory Breakdown Analyzer:** Inspect how many bytes each data structure consumes (identify memory hogs instantly).
- **Built-in CLI Workbench:** Test raw Redis commands directly in the browser with full autocompletion and command syntax hints.

7. Connecting via Terminal CLI (Sanity Check)

To verify the setup from your terminal without installing the native `redis-tools` package:

```
docker exec -it redis-cache redis-cli -a devpassword123
```

Once inside the interactive prompt, test responsiveness:

```
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SET test:ping "hello world"
OK
127.0.0.1:6379> GET test:ping
"hello world"
127.0.0.1:6379> EXIT
```

With the infrastructure running and visually monitored, the development environment is ready to explore core Redis commands and data structures.